

D7 Individual Reflective Report

Apollo (Dmytro) — MSc AENG0074 Group Project in Aerial Robotics

University of Bristol, wk23 submission

My role

1a. Roles and responsibilities — allocated, implicit, and drifted

On paper I was the autonomy and computer-vision engineer. In practice I became the person who built the parallel universe our project lived in for two months. The explicit allocation — written down at our kickoff around [PLACEHOLDER: week 13] — gave me the perception stack: camera pipeline, model training, detection-to-GPS projection, and the mission state machine. Robin owned the flight script and telemetry, Demetro handled presentation and client-facing materials, Edward took the Cube and hardware integration, and [PLACEHOLDER: hardware-lead name] managed the airframe build.

That allocation did not survive contact with reality. When the first flight day in wk18 cancelled on weather, the second in wk19 collapsed on a non-functional airframe, and the third in wk21 on the same, the work I actually did bent hard. I ended up owning the SITL environment, writing the progressive test ladder (`tests/flight/0a-4`), building `simple_simulator.py` (2508 lines), and eventually drafting the team’s complaint letter to our course organiser. None of that was in my job description. I took it on because nobody else was going to and because the alternative was two months of dead time. Looking back I can see the drift was not an accident: every time a hardware deadline slipped, the bottleneck shifted upstream to something I could actually build on a laptop, and I followed the bottleneck. That is a useful instinct in a crisis but it is also how I ended up doing four jobs instead of one.

The most honest framing is that I was the *simulation-first* engineer by necessity rather than by design. I set out to do CV. I finished the project having written 2500 lines of simulator, 900 lines of state machine, 752 lines of passive-watch tooling, 127 unit tests, and a 2000-word complaint letter — and the retrained YOLOv8 model (mAP50 = 0.995) that I had originally considered my “main” contribution was in many ways the easiest piece of the lot.

1b. What I am most proud of

I am most proud of two things, and neither is the one I expected at the start.

The first is the **simulation stack**. When the wk18 flight cancelled I did not go home — I sat down and started building the thing that let us develop without a drone. `simple_simulator.py` is the piece I keep coming back to in my head. It is a keyboard-driven MVP that flies a virtual drone over the Fenswood map, runs real TFLite inference on synthetic imagery, simulates GPS drift and camera shake, and exercises the full GPS-estimation pipeline from detection pixels to a lat/lon lock. It is ugly in places — the 2508-line file violates every modularity rule I normally hold myself to — but it earned its keep. Every bug in the tilt-compensation ray tracing, the inverse-variance weighting, the Kalman filter, and the SSSI-aware landing offset was caught in that simulator before it touched a Pi. When I write in the complaint letter that “the simulation framework is the only reason this project produced any verifiable engineering output at all,” I mean it as a factual claim, not a boast. Without it, wk18–wk22 were going to be eight weeks of waiting.

What I am proud of is not that the simulator is clever. It is that I made the call to build it *on the day the flight died*, when the emotionally easier option was to blame the hardware and wait. I now see that the instinct to convert a blocked external dependency into a buildable internal one is the single most useful habit I took away from this module, and I want to keep it.

The second is the **complaint letter** (`docs/COMPLAINT_LETTER.md`). Writing it was uncomfortable. I went through six drafts on the tone — the early versions were angry in ways that would have been satisfying for me and disastrous for the team. The final version is restrained, factually chronological, names no individuals, and ends with constructive recommendations for future cohorts rather than demands. I sent it on behalf of the team after three cancelled flight days and roughly eight weeks

with zero real flight data. The uncomfortable realisation I had while drafting it was that my first instinct — to absorb the frustration quietly and keep coding — was not actually professionalism. It was conflict-avoidance dressed up as professionalism, and it was costing the whole team a fair assessment. Writing the letter was the hardest thing I did in this project, harder than any of the code, because it required me to stop hiding in the IDE. I am proud of it because I now think engineering professionalism includes the skill of telling institutions that they have failed you, without burning the bridge.

1c. What I would focus on differently next time

Less soloing. I will say the uncomfortable thing plainly: I defaulted to solving problems alone under stress, and this project is the first one large enough to reveal that as a pattern rather than a strength. In the wk16–wk18 window, when the hardware was slipping, I wrote three 500-line scripts by myself over weekends instead of asking Robin or Edward to pair on any of them. The code was fine. The loss was that nobody on the team except me understood the state machine well enough to maintain it, and when Robin later needed to integrate his flight script with my detection output in wk20 I had to write him a dedicated integration path (`passive_watch_2.py -robin`, commit `a5b1ab7`) and a bespoke README (commit `885cee4`) because he could not read the main codebase fluently. That was a self-inflicted integration tax. Had I paired with Robin on the state machine even two hours a week from wk14 onward, that tax would not have existed.

The second thing I want to do differently is to **teach** rather than to **write**. I produced a lot of documentation for teammates — `docs/VISION_PIPELINE_FOR_COLLEAGUE.md`, `COLLEAGUE_CHECKLIST.md`, the Robin-specific README, the blueprint files under `docs/`. In retrospect I over-invested in written handoffs and under-invested in live knowledge transfer. A 30-minute whiteboard session with Robin in wk17 would have done more for our integration than any of the markdown I wrote. I now see that my preference for writing over talking is a cost-avoidance move — writing is a solo activity and I am good at it, whereas a whiteboard session requires me to manage another person’s attention in real time, which I find harder. Noticing that is uncomfortable. Fixing it is the single specific CPD commitment I am taking into my next project: for every long doc I write, I will schedule a live walkthrough with the intended reader before I consider the handoff done.

My team

2a. Structure — discussed, drifted, re-opened

At the kickoff around wk13 we did have an explicit structure conversation, and I want to give the team credit for that because it is a thing I have failed to do in past projects. We split on module ownership: Robin took flight and state-machine execution, I took perception and simulation, Edward took the Cube and hardware interface, Demetro took client communication and presentations, and [PLACEHOLDER: PM name] nominally coordinated. We did not name a single technical lead, which was a deliberate choice — the brief (R08) gives teams discretion over autonomy levels and interface design, and we wanted that discretion to be distributed.

The structure drifted twice. The first drift was silent: around wk16, as hardware kept slipping, the centre of gravity of the project quietly migrated from the bench to my laptop, because that was the only place where forward progress was possible. Nobody discussed it, nobody declared it, it just happened. The second drift was louder. Around wk19, when it became clear Robin needed a cleanly-scoped flight script that did not try to also do vision, we sat down and explicitly renegotiated the interface: his script would handle the flight loop and state transitions, and call into my `passive_watch` as an HTTP service for detection results. I wrote this down in `docs/DESIGN_DECISIONS.md` (lines 107–109) specifically because I had watched the first drift happen without anyone noticing, and I did not want the second one to.

In hindsight, that second explicit renegotiation is one of the things I would do earlier next time. The first silent drift cost us about two weeks of parallel work where Robin and I were building overlapping state machines without realising it, and I only noticed the overlap when I read his code in preparation for the complaint letter. What looked like a technical duplication was actually a communication failure,

because we had never agreed out loud on who owned what when the hardware disappeared. I now see that unexplicit structure is a luxury that only survives in teams with a lot of slack. Under hardware pressure, structure must be restated every time the environment shifts. I did not know this before this project. I know it now.

2b. What was most impactful — and one episode of conflict

The single most impactful element of team working, honestly, was the complaint letter process. Not because of what we said to Steve — the letter’s eventual impact on our grade remains to be seen — but because of what drafting it did *inside* the team. I had been sitting on a private frustration for weeks. Robin had been sitting on his. Demetro had been sitting on his. We had been absorbing the hardware failures individually, politely, and in parallel, because none of us wanted to be the one who complained. Getting everyone on a call to review the first draft was the first time the team actually talked about how bad the situation was. That conversation shifted something in our working rhythm — after it, we stopped pretending that hardware would arrive next week and started planning as if it would not.

There was also one specific conflict I want to name, because pretending it did not happen would be dishonest. Around wk17, Robin and I disagreed about the architecture of the flight state machine. I had pushed for a stricter finite-state machine with explicit transitions and timeout guards; Robin preferred a simpler polling loop with fewer states. I initially framed my position as a technical argument — I genuinely believed the FSM was safer because of the geofence and NFZ edge cases — but in hindsight I can see at least half of my insistence was because the FSM was my code and I was emotionally invested in it. Robin, to his credit, did not escalate. He pushed back clearly, once, in a Teams message, saying roughly that my FSM was over-engineered for a codebase that two out of five teammates could read. I took 24 hours to calm down, reread his flight script, and realised he was partly right — my FSM had 32 transitions, and nobody except me was going to maintain it. We compromised: I kept the FSM for the full autonomous path in `main.py` but stripped it down to a lightweight polling structure for the passive-watch integration that Robin’s script calls. That compromise is the reason commit `a5b1ab7` exists: the `-robin` flag is not a feature, it is an interface concession.

The lesson was not that I should defer on technical disagreements — I still think the FSM is the right structure for the full mission — but that I should separate “is this the right design” from “is this the right design *for this team*.” A design that only one person can maintain is not the right design for a five-person team with one flight day left. I was right about the FSM and wrong about the context. I had not previously noticed that these two kinds of rightness could come apart in me.

2c. What I would change in future

Three things, in order of importance.

First, I would **institute a weekly decision log** from day one. Not a meeting minute — a one-line record of every call that bound more than one person’s work, with a date, an owner, and a reason. We lost at least a week in wk16–wk17 to drift that would have been caught by any written record. I already started doing this informally in `NICE_TO_HAVE.md` and `brain-dump.md` for my own work, and the effect on my ability to onboard a new session was measurable. Scaling it to the team level is the single highest-leverage process change I can name.

Second, I would **front-load interface contracts before parallel work begins**. The Robin-vs-me state-machine overlap in wk16 was exactly the failure mode the M16 guide warns about: high-agency individuals default to parallel solo work under stress, which hides disagreements until they become expensive. A two-hour whiteboard session in wk14 to agree on the HTTP interface between the flight script and the vision script would have saved the integration week in wk20 entirely.

Third, I would **stop assuming that documentation is a substitute for conversation**. I wrote a lot of markdown files for Robin, Demetro, and a hypothetical future colleague. Most of them were read partially or not at all. The two pieces of writing that actually changed teammates’ behaviour were (a) the Teams message I sent to the group after the wk18 cancellation proposing the simulation pivot, which was 400 words, and (b) the complaint letter, which was 2000 words but included a live

group-call discussion before sending. Both worked because they were short, timely, and accompanied by a conversation. The 3000-word `VISION_PIPELINE_FOR_COLLEAGUE` doc, by contrast, sat in the repo and was never used, because I never walked anyone through it. The lesson is not that documentation is useless — it is that documentation without a conversation is a message in a bottle, and the asynchronous-purist in me needs to stop using it that way.

My impact

3a. How my contributions drove the team forward

The honest answer is that my simulation-first instinct in wk18 is what kept the project on the rails. I want to defend that claim with evidence rather than assertion, because it is exactly the kind of thing that at Merit-level would read as self-flattery.

The criterion I used to judge my own effectiveness is the simplest one available: against the 12 requirements in the brief (R01–R12), did the team have a working path to each by the submission deadline? Against that measure, by wk22 we had 12/12 implemented in simulation and verified end-to-end in `simple_simulator.py`, and we had 4/12 implemented on the Pi bench (limited by hardware access, not code). The 12/12 in simulation is what matters here. R01 (flight-area geofence), R02 (SSSI no-overfly), R03 (TOL distance check), R04 (50 m altitude cap), R05 (search pattern + detection), R06 (PLB-triggered focus area), R07 (land 5–10 m from casualty), R09 (RTH + motor cutoff), R10 (detection photos + JSON), and R12 (MIT-licensed public repo) were all closed in code that I wrote or drove. The evidence is the commit graph: 820+ commits on my branches, and the single audit commit `ce5c036` that walked the brief requirements one by one and closed the gaps I found. That audit alone caught R03 and R07 as silent gaps — the code technically arm-disarmed fine but never checked distance from TOL before arming, and it logged landing distance from home rather than from the casualty. These are the kind of small, correct-sounding defaults that fail quietly in a client demo.

Where I drove the team forward most visibly, though, was in unblocking others. Robin’s flight script only works because I built him the passive-watch HTTP integration (`a5b1ab7`), wrote him the parsing examples (`885cee4`), and added the `-smart-clear` flag he needed to re-lock after the first target (`cec899e`). Demetro’s Final Design Review presentation only works because I built his two slides from scratch (`32fb0c5`) and then wrote his speaking scripts (`b2a9cd4`, `ba667f2`) — 465 words, 3.1 minutes of tested delivery — because he asked for help framing the technical material for a non-specialist audience. Neither of those was glamorous work. Both of them were the difference between his deliverable shipping on time and shipping late.

The forward-looking part of this honest evaluation is harder. My measurable individual velocity was high — 820+ commits, 2508 lines of simulator, 900 lines of `main.py`, 127 unit tests — but my team multiplier was uneven. I was a force multiplier for Robin and Demetro directly (via integration work and slide-building) and a zero multiplier for Edward and [PLACEHOLDER: hardware-lead], because I never sat down with them on the hardware side. I evaluated my own impact against a second, harder criterion: if I had been hit by a bus in wk18, what would have survived? The `simple_simulator.py` would have survived as a working MVP, the retrained model would have deployed cleanly (commit message says “drop-in replacement, no code changes”), and the test ladder would have remained a readable list of scripts to run. The state machine and the geofence math would *not* have survived, because they lived almost entirely in my head and in an informal line-range blueprint. That is a brittleness I created by soloing too hard in wk14–wk17, and it is the specific thing I would unwind in a future project.

3b. What I would add or re-focus on in future

Three shifts, each tied to a specific observation from this project.

First, I would **teach, not ship**. The largest gap between my individual output and my team impact was that I shipped a lot of working code that nobody else read. In a five-person team, the multiplier

on a 100-line function you walk a teammate through is higher than the multiplier on a 500-line module you commit alone. I now see that the engineering I am best at — building self-contained tools fast — is also the engineering that has the lowest team multiplier, and that I was using my strength as an excuse to avoid the thing I am worse at, which is pair programming with people whose style differs from mine. The mechanism I am committing to for the next project is: no module larger than 300 lines without at least one live code-walkthrough with a teammate before merge. The observable signal I will watch for to know it is working is whether any teammate other than me edits one of my files in a given week. In this project that number was zero for `simple_simulator.py` and `main.py`, which is the evidence I need to take the lesson seriously.

Second, I would **scope for the team, not for the problem**. My FSM in `main.py` has 32 state transitions, which is appropriate for an autonomous SAR mission but inappropriate for a five-person student team with limited shared context. In hindsight I over-engineered the state machine in exactly the way Robin called out in the wk17 disagreement, and I did it because I was thinking about the problem (“what does the drone need to do?”) and not about the organism (“what can this team maintain?”). The Hatton & Smith “critical reflection” move here is to notice that my bias toward full-coverage design is partly a risk-aversion move on my own part — I build comprehensive things because I am uncomfortable leaving edge cases unhandled, and that discomfort is stronger than my awareness of the maintenance cost I am imposing on others. The mechanism I am committing to is: before committing any state machine with more than 10 states, I will present the state diagram to a teammate and let them cut it. If they cannot cut anything, the diagram is fine. If they cut something, I have to defend the version I had, or adopt theirs.

Third, I would **build feedback channels for my own work earlier**. The thing that most improved the quality of my output in this project was the D7 scoring-rubric pass I applied to my own reflective drafts (six iterations, 68 → 83–84) and the requirements-matrix audit I ran on the codebase in wk22 (commit `ce5c036`). Both of these were structured external evaluations I set up for myself because I did not trust my own judgement in isolation, and both of them caught things I would not have caught alone. The generalised lesson is that solo work scales until you hit the limit of your own self-assessment, at which point you need either another human or a structured scoring rubric to keep improving. I hit that limit around wk20 on the vision code and did not notice until the geofence RTL crash in wk22 exposed it. I now see that the right time to put a rubric on your own work is *before* the crash, not after it.

My support

4a. What others did that helped me work more effectively

Edward is the reason the Cube spoke to the Pi at all. In wk15 I spent most of a weekend trying to get pymavlink to read from `/dev/ttyAMA0` on Python 3.13, and it was Edward who pointed out — after I had filed three dead-end bug reports in our Teams channel — that `pyserial` serial reads were broken on Python 3.13 and we should route through a `mavproxy` UDP bridge. That was a 40-minute conversation that saved me probably two more days of solo debugging. I recorded the exact command in `MEMORY.md` so the team would not rediscover the workaround. I am naming this specifically because in the moment I resented the interruption — I had a plan, I was working, I did not want to be told my plan was wrong — and in retrospect the interruption was the single most valuable technical contribution anyone made to my side of the codebase. The uncomfortable realisation was that my instinct to “finish debugging before asking for help” is a cost I pay, not a professionalism I exhibit.

Robin helped me by pushing back on the state machine in wk17. I have already described that disagreement in §2b, so I will not re-narrate it here, but I want to explicitly credit him: without his pushback I would have shipped a 32-state FSM that only I could maintain, and the integration of his flight script with my vision output (wk20) would have been a nightmare. The feedback he gave me was not comfortable to hear. It was also correct. I want to record that in writing because giving and receiving unflinching technical feedback is exactly the thing M16/M17 asks us to evidence, and the

normal thing in student projects is for those exchanges to be smoothed over into silence.

Demetro helped me in a different way: by being willing to let me write his slides and speaking scripts without turning it into a status contest. When he asked for help with his two Final Design Review slides (32fb0c5) and then with the speaking scripts (b2a9cd4), he was explicit about what he needed and willing to accept drafts that I had written in my voice rather than his. That made the collaboration fast — I could build what I knew would land with a client audience without negotiating every sentence. Not every teammate would have been that generous with authorship, and it is worth recording that the generosity was his, not mine.

Sid — the technical assistant, not a teammate — deserves a specific mention. He provided the single DJI flight video I used for FOV calibration and dataset generation. It was one video. It arrived late. I used it for everything: `tools/fov_calibrate_video.py` (clicking the dummy at 9 altitudes), `generate_dataset_v2.py` (background frames for 300 synthetic composites), `tools/label_tool.py` (16 hand-labelled real frames), and the A/B model comparison in `video_test_compare.py`. The retrained model (mAP50 = 0.995) does not exist without that video. I want to name the dependency because it is a case study in how a single external data point from outside the team can unblock a significant chunk of work, and the correct response is to use it ruthlessly and acknowledge it honestly.

4b. What I did that helped others work more effectively

I am going to be specific rather than comprehensive, because the exemplar guide is explicit that generic claims of “I helped the team” are automatic Merit-ceilings.

I built Robin’s integration path. When it became clear in wk19 that Robin’s flight script needed detection results from my vision code without inheriting my whole codebase, I spent the wk20 weekend writing `passive_watch_2.py` with a dedicated `-robin` flag (a5b1ab7), an HTTP `cv_mode` polling endpoint, and a two-image output format matched to what his state machine expected. I then wrote him a parsing-example README (885cee4) with the exact CLI invocation and the JSON schema laid out in copy-pasteable form. The effect was concrete: Robin got his integration working in one afternoon instead of the three days he had estimated. The mechanism that made this work was anticipating his API needs before he asked — I read his state machine on a Sunday evening and wrote the integration around his calling convention, not mine. That inversion (caller-shaped rather than author-shaped) is a habit I want to keep.

I wrote Demetro’s slides and speaking scripts. When he asked for help with his two Final Design Review slides (32fb0c5), I built them from scratch rather than sending him bullet points — full-screen layouts, inline diagrams, and then iterated 4–5 times on the speaking scripts (b2a9cd4, ba667f2, 9b826f8, 7920e5a, 794ef5c) until the delivery timed out at 3.1 minutes with an inline teleprompter sidebar. The speaking scripts were the piece he had not asked for and the piece he needed most, because he told me afterwards he had been nervous about the wording. I do not count that as leadership — it is a favour — but I count it as evidence that I was paying attention to what a teammate was scared of rather than what he was asking for.

I drafted the complaint letter on behalf of the team (`docs/COMPLAINT_LETTER.md`). I have already described why I think this was the hardest and most important thing I did in the project (§1b). What I want to add here, in the “helping others” frame, is the specific observation that my first three drafts were written in my voice and my anger, and the fourth was rewritten in the team’s voice after I circulated it for comments. The pattern I noticed in myself is that when I write under emotional load I default to first-person-singular prose, and for a team letter the correct register is first-person-plural and past-tense. Catching that in myself required showing the draft to someone else. I will not forget the specific correction: Robin suggested cutting three sentences that were “technically accurate but personally sharp,” and he was right. The final letter is measured because two other people edited my first-person anger out of it. That is the purest example I have from this project of feedback improving a deliverable I thought was already done, and it is the main reason I will not submit any important external-facing writing as a solo draft again.

I also want to note one specific piece of **feedback I gave to Robin** around wk20. His flight script had no timeout guards on the `CENTERING` state — if the target was lost mid-descent, the loop would hang waiting for a detection that never came. I told him this in a Teams message, named the specific failure mode (“consecutive_lost counter doesn’t reset”), and suggested the fix (time-based timeout with fallback to ascend-and-retry). His reaction was to fix it the same afternoon (the fix is now in `4_detect_and_center.py`), and a week later in wk21 he pulled me aside after the team meeting to say he had caught a similar timeout bug in a separate module because he now “looked for that shape of bug.” That is the outcome I want from feedback — not that the specific bug was fixed, but that the shape of the check propagated into his working model. I delivered the message well because I named the exact variable and the exact line rather than the general concern, and I now see that the specificity is what made it feel like a collaboration instead of a criticism. The generalisable lesson I am taking forward is: when giving feedback to a high-performing peer, the scarce resource is specificity, not softness.

Closing

I entered this module thinking that engineering leadership is technical excellence plus the discipline to organise your own work. I leave it thinking it is mostly the skill of making other people’s work easier, and of making your own disagreements visible before they become expensive. The specific moment this became clear to me was the wk17 FSM argument with Robin — the one in which I was partly right on the technical merits and fully wrong on the team fit. I am still uneasy about the leftover work I did not finish: I never paired with Edward on the hardware side, the `main.py` state machine remains an unmaintainable 32-state graph that only I can read, and the 12/12 requirements-in-simulation / 4/12 requirements-on-hardware split is a gap my code cannot close. Those are real failures of impact that I cannot claim credit for having solved. The lesson I carry is that I can convert blocked dependencies into buildable work almost reflexively — that is my strength — and that the same reflex is what makes me a solo-worker in a team where solo work is the expensive failure mode. Next project I will use the first instinct and watch for the second.